

Contents

1	Installing LaTeX efficiently	2
2	Compiling a LaTeX file	4

1 Installing LaTeX efficiently

This section is solely for those people who would rather not install around $10GB$ of $\LaTeX$ locally, and are users who have and want to have little to do with $\TeX$ outside the scope of the solver. Regardless of this, you will still have to compile and read the `UserGuide` after going through this `InstallationGuide`. Do not worry about the technical $\LaTeX$ terminology in this document, all of it has been explained in the `UserGuide`.

A complete installation of $\LaTeX$ as provided by `texlive` contains everything that you could ever require, but the solver obviously does not need all of it. If one were to cut down on the superfluous chunks of software, the resultant $\LaTeX$ installation takes up around $4.4GB$ of space. What you save in space, you gain in the difficulty of the initial setup. Weigh your options well.

`Texlive` offers installations for both Windows and MacOS, and although I have tested only the Windows version, the necessary collections and packages remain the same regardless of operating system. The `texlive` installer offers various ‘schemes’ in its advanced settings, which are predefined installation settings created for users, best suited for particular purposes. We select the ‘custom’ scheme, and setup all necessary settings ourselves.

All related and dependent packages are grouped under a ‘collection’, which are self-contained, and are most sensibly installed as one unit. Under the ‘custom’ scheme, one may select various collections to install initially. The collections to be chosen are:

<i>Name</i>	<i>Description</i>
basic	Essential programs and files
latex	$\LaTeX$ fundamental packages
latexextra	$\LaTeX$ additional packages
latexrecommended	$\LaTeX$ recommended packages
pictures	Graphics, pictures, diagrams

Do not choose any languages, these will be added later with some other collections. On successful installation of $\LaTeX$, the wizard can be closed.

To proceed further, the `tlmgr` tool is required. `tlmgr` is a package management tool that allows easy addition/removal of packages and collections after the initial installation. It is launched using the ‘command prompt’ or ‘powershell’ by using the code:

`tlmgr (Command) (additional argument) (package/collection name)`

Run `tlmgr` with ‘Command = install’, ‘additional argument =’ and the package/collection names as follows. Note that every collection name follows `collection-`:

Collections:	
<i>Name</i>	<i>Description</i>
<code>langchinese</code>	Chinese
<code>langcyrillic</code>	Cyrillic script languages
<code>langenglish</code>	UK and US English
<code>langgerman</code>	German
<code>langjapanese</code>	Japanese
<code>langkorean</code>	Korean
<code>langotherv</code>	Other languages
Packages:	
<code>ulem, latexmk, lua-calendrica</code>	

These can be installed simultaneously, provided their names are separated by a space:

```
tlmgr install collection-langchinese collection-langotherv ulem
```

The above command will install and configure the given collections/packages. After the configuration is complete, the UserGuide can be generated, and the solver be run in theory.

Note: Installing the collections for either of chinese/japanese/korean automatically installs the collection `langcjk`, which they depend upon. Although users need not explicitly worry about this, do not remove this collection just because it is not mentioned in the above table.

One may wonder about why so many languages are required for a basic compilation. The reason is simply that I did not write the solver with compact installations in mind, since as time passes, the size of the solver is expected to keep increasing. Aside from these basic language collections, one may choose to selectively install only those collections which are of interest. Keep in mind that aside from the `\include` line in

ENG.tex, the `\setotherlanguage` command in package.tex must now be similarly commented out to eliminate unwanted language solutions in the main file, since the corresponding language collection won't be installed and would cause a polyglossia compilation error.

Font definitions need not be touched, since `\(script)font{}` depends on fontspec, not on polyglossia language switching.

2 Compiling a LaTeX file

Depending on preference, users of $\text{\LaTeX}$ may use multiple methods of running it on a file. I will document three such methods here, but the user is encouraged to mess around with others as well:

1. Console commands
2. VS Code
3. $\text{\TeX}$ Studio

Console commands are for people who are more machine than human, and don't care about beauty. To use them, you use 'powershell/command prompt' with

```
cd("(File directory)")
```

and point to the directory where the '.tex' source file resides. Then, one may run

```
(latexengine) (additional argument) (File name).tex
```

to start compilation. If an error occurs, the '.log' file must be checked and the errors corrected. Using this method, it is possible to edit and compile using no more than traditional '.txt' files and the console. An example using `lualatex` would be:

```
lualatex --shell-escape ENG.tex
```

These commands are simple, and best suited for users who just want the solver to work, and have no interest in editing it further.

Alternatively, one may use the `latexmk` tool and run

```
latexmk -lualatex --shell-escape ENG.tex
```

A source file normally requires multiple reruns before the final pdf is usable. This happens because of things like cross referencing, which cannot be handled in a single forward pass. The user must be aware of this if using base `lualatex` commands, and run $\LaTeX$ multiple times until the output is correct. `ENG.tex` needs two complete ‘`lualatex`’ compilations to generate a usable pdf: do not be surprised or worried if the output pdf looks incomplete or broken after a single run. `latexmk` on the other hand handles all completeness checks internally and need only be triggered once.

VS Code is a code editor that offers a middle path between beauty and functionality. Using the tools provided by the $\LaTeX$ extension, it is possible to easily edit and debug source files. The extension also provides a clean interface for common rerun patterns when compiling a source file, which are termed as recipes. Using the ‘`latexmk (lualatex)`’ recipe, the user can compile with a single command, without worrying about rerun details. However, VS Code is ultimately a general code editor, and retains some console like properties.

T_EXStudio is a graphical user interface (GUI) created solely for the purpose of editing and compiling ‘.tex’ files. This is the best alternative for someone who wishes to update and use the solver but doesn’t want to deal with the headache of troubleshooting code. Everything has been neatly sorted and made intuitive enough, so that beginners do not get overwhelmed with unknown commands and their interpretations.

Irrespective of the method used, the main file `ENG.tex` must be run using `lualatex --shell-escape` for every run which introduces changes in the vector image pdf(s) generated by the `tikz` package (stored in folder ‘figures’). If no changes are made, `--shell-escape` is not necessary.

ओजस अमित सेवेकर - langsolve-en